



This study guide demonstrates the lesson from *Create Highly Available Applications with Load Balancers*

My full AWS Architect Associate course can be found here:

<https://www.udemy.com/course/ultimateaws/?referralCode=7ED214B795C444141361>

Understanding Elastic Load Balancers (ELB) and Network Load Balancing in AWS Study Guide

In this lesson, we will explore the basic concepts of Elastic Load Balancers (ELBs) in AWS, focusing specifically on **Network Load Balancers (NLBs)**. Load balancing is an essential practice in cloud computing, enabling traffic distribution across multiple instances for scalability, availability, and fault tolerance. While this lesson focuses on NLBs, the concepts also apply broadly to other AWS load balancers, such as Application Load Balancers (ALBs) and Gateway Load Balancers (GLBs).

Introduction to Load Balancers

A **load balancer** acts as a traffic manager, distributing incoming requests across multiple backend targets, such as EC2 instances, containers, or Lambda functions. Load balancers improve:

1. **Scalability:** Supporting more traffic by distributing workloads across multiple instances.
2. **Availability:** Ensuring high uptime through redundancy and health checks.

AWS Elastic Load Balancing provides three types of load balancers:

- **Network Load Balancer (NLB):** Optimized for high performance and low latency, suitable for TCP and UDP traffic.
 - **Application Load Balancer (ALB):** Operates at the application layer (Layer 7) and is used for HTTP/HTTPS traffic.
 - **Gateway Load Balancer (GLB):** Facilitates transparent deployment of third-party virtual appliances.
-

Network Load Balancer Basics

Definition:

A **Network Load Balancer (NLB)** operates at Layer 4 (the transport layer) of the OSI model, using IP addresses and ports to manage traffic. It is ideal for high-performance workloads and scenarios requiring low latency.

Key Features:

1. **Traffic Distribution:** Routes incoming traffic to backend instances across multiple **Availability Zones (AZs)**.
 2. **Health Checks:** Continuously monitors the health of targets and ensures traffic is only routed to healthy instances.
 3. **Scalability:** Handles millions of requests per second, making it suitable for high-traffic applications.
 4. **Static IPs:** Each NLB provides a static IP address per AZ, simplifying DNS management.
-

Benefits of Using a Load Balancer

1. Horizontal Scaling (Scaling Out)

- Instead of relying on a single large instance, multiple smaller instances are deployed to handle traffic.
- Horizontal scaling increases fault tolerance: if one instance fails, others can continue serving traffic.

2. Improved Availability

- Load balancers detect unhealthy instances using periodic **health checks**.
 - When an instance or AZ becomes unavailable, the load balancer automatically stops routing traffic to it.
-

How a Network Load Balancer Works

Scenario Overview:

- Imagine a website hosted on multiple EC2 instances, distributed across two availability zones (AZs).
- The NLB evenly distributes traffic across all healthy instances in both AZs.

Target Groups:

- Backend instances connected to the NLB are organized into **target groups**.
- A target group specifies:
 - Protocol and port for traffic (e.g., TCP on port 80).
 - Health check configurations (e.g., ping frequency, response timeout).

Traffic Flow:

1. Incoming traffic reaches the NLB.
 2. The NLB forwards requests to EC2 instances in the target group.
 3. Traffic is distributed across healthy targets within multiple AZs.
-

Health Checks

A **health check** is a periodic test performed by the load balancer to ensure targets are functioning correctly. If an instance fails a health check:

- The NLB stops routing traffic to it.
- Traffic is redirected to healthy instances.

Example:

- If an entire AZ fails, the NLB identifies unhealthy instances and routes all traffic to the remaining healthy AZ.
-

Handling Failures

Example Scenario:

1. **Normal Operation:**
 - Six EC2 instances are active: three in AZ1 and three in AZ2.
 - The NLB distributes traffic evenly across all instances.
 2. **Failure Scenario:**
 - AZ2 fails, taking its three instances offline.
 - The NLB detects this via health checks and stops sending traffic to AZ2.
 - Traffic is routed only to the three healthy instances in AZ1.
 3. **Auto Scaling:**
 - If **Auto Scaling** is enabled, additional instances can be launched in AZ1 to handle the increased load.
-

Horizontal Scaling vs. Vertical Scaling

Horizontal Scaling:

- Adds more instances to handle increased traffic.
- Advantages:
 - Higher fault tolerance: failure of one instance doesn't affect others.
 - Easier to adjust to varying workloads.

Vertical Scaling:

- Increases the size (e.g., CPU, memory) of a single instance.
- Disadvantages:
 - Single point of failure.
 - Limited scaling capacity.

Horizontal scaling, supported by load balancers, is preferred in modern cloud architectures.

Configuring a Network Load Balancer

Steps to Set Up an NLB:

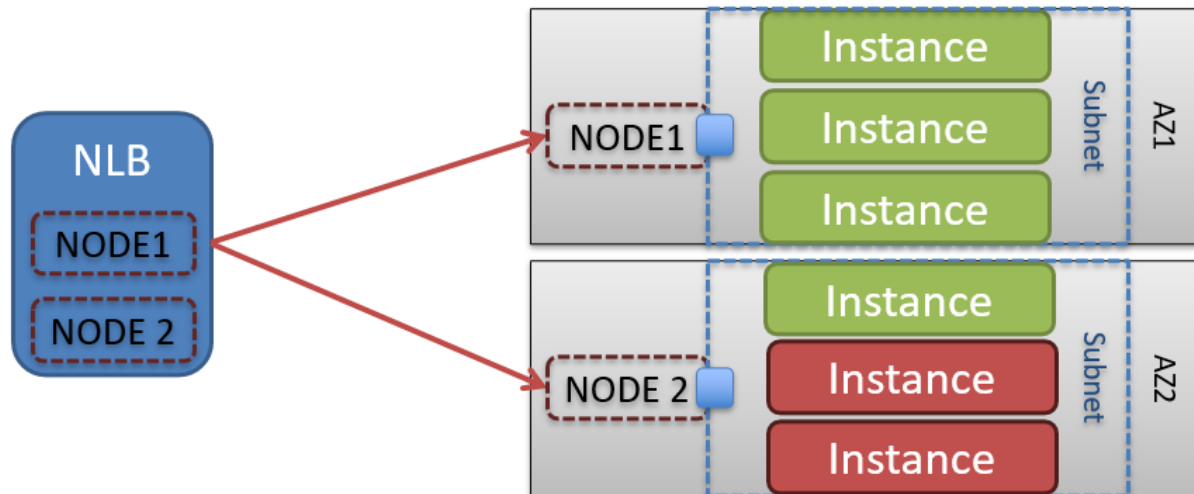
1. **Access the AWS Console:**
 - Navigate to the **EC2 Dashboard** and select **Load Balancers**.
 2. **Create a Network Load Balancer:**
 - Select the NLB option.
 - Assign a name (e.g., "WebApp-NLB").
 - Choose the VPC and availability zones for your targets.
 3. **Configure Listeners:**
 - Specify the protocol and port (e.g., TCP on port 80).
 4. **Create Target Groups:**
 - Add backend EC2 instances to the target group.
 - Define health check parameters.
 5. **Test:**
 - Use the NLB's DNS name to send traffic and verify distribution across instances.
-

Summary

- **Elastic Load Balancers** are critical for distributing traffic, ensuring high availability, and enabling horizontal scaling.
- **Network Load Balancers** are optimized for high-performance, low-latency workloads.
- Key features of NLBs include static IPs, health checks, and multi-AZ traffic distribution.
- Horizontal scaling with load balancers provides fault tolerance and better resource utilization compared to vertical scaling.

See slide below:

Network Load Balancer Example



For more details see my full AWS Architect Associate course:

<https://www.udemy.com/course/ultimateaws/?referralCode=7ED214B795C444141361>